

Systems Verification

Exercise Sheet 06 - Solutions

Exercise 1: From never claim to LTL formula (optional)

Suppose that we consider some model that contains the following never claim, where p is some atomic proposition.

```
never {  
  T0_init:  
    do  
      :: ((p)) -> goto accept_S9  
      :: (1) -> goto T0_init  
    od;  
  accept_S9:  
    do  
      :: (1) -> goto T0_init  
    od;  
}
```

Describe what is the property that we want the *model verifies* and give the LTL formula that is satisfied by the model.

Solution: In this never claim we can visit the accept label only when p is true. Since we have to visit the accept label infinitely many times in order to make the never claim successful, the formula satisfied by the never claim is $\Box\Diamond p$ (i.e. p is true infinitely many times). The property we want the model to satisfy, is the *negation* of the property that makes the never claim successful, therefore we are verifying whether the model satisfies

$$\neg\Box\Diamond p \equiv \Diamond\Box\neg p,$$

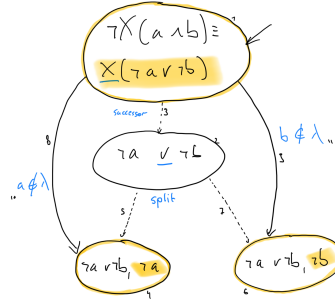
which in English can be stated as " p is true a finite number of times".

Exercise 2: Constructing SLBA

Using the construction seen in class, construct the *SLBA* for each of these formulae and the corresponding eventuality automaton if necessary:

1. $\neg \mathcal{X}(a \wedge b)$

Solution: Using the equivalence seen in class for $f\mathcal{X}g$, we can process the initial formula and we obtain the following SLBA, keeping in mind that a $\mathcal{X}$ creates a successor state and a disjunction creates two new states:

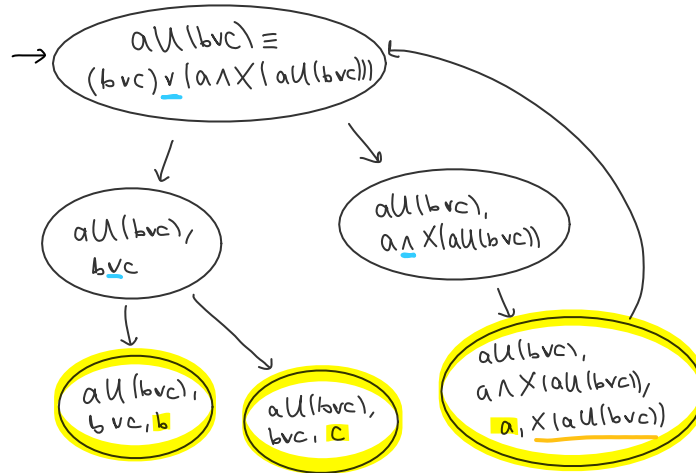


States highlighted in yellow are the *real states*, since they contain an atomic proposition and can't be processed any further. The next step is to build another SLBA, where only the real states of the first one are kept and the transitions are added in such a way to allow the same change of states as in the preceding SLBA. All the states are accepting.

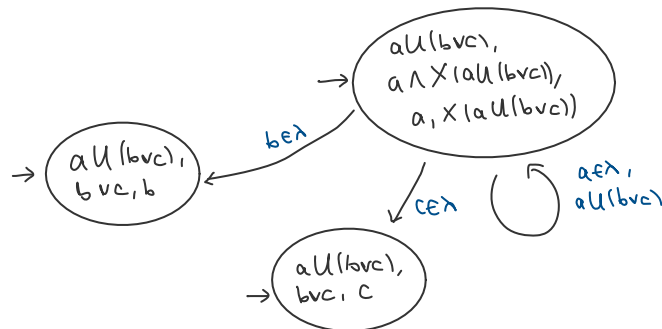
Therefore we remove the transitions numbered 3, 5 and 7 since they go to or come from an intermediate state. We then add transitions numbered 8 and 9 to allow the same change of state. These transitions are then labeled according to the atomic propositions of the target states.

2. $aU(b \vee c)$,

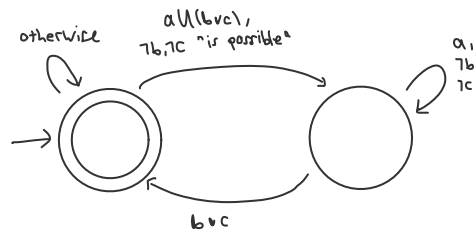
Solution: Using the equivalence seen in class for fUg , we can process the initial formula and we obtain the following SLBA, keeping in mind that a disjunction creates two new states:



States highlighted in yellow are the *real states*, since they contain an atomic proposition and can't be processed any further. The next step is to build another SLBA, where only the real states of the first one are kept and the transitions are added in such a way to allow the same change of states as in the preceding SLBA. All the states are accepting.

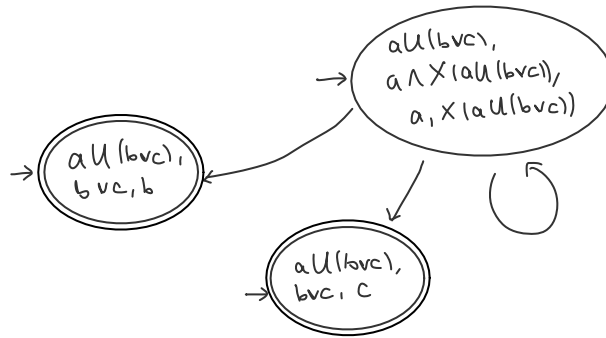


Since the formula contains an unnegated until sub-formula, we have to consider the following eventuality automaton):



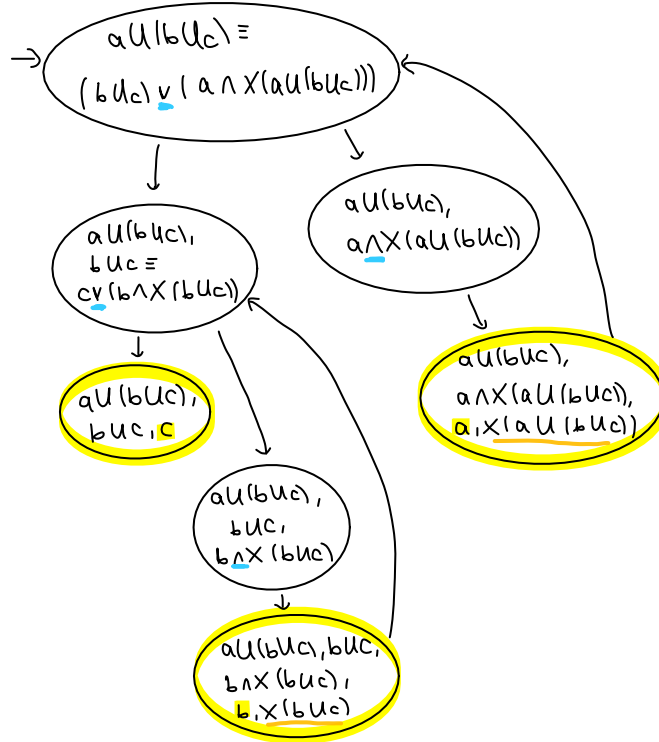
And finally we obtain the following SLBA, by intersecting the real states SLBA with the

eventuality automaton:

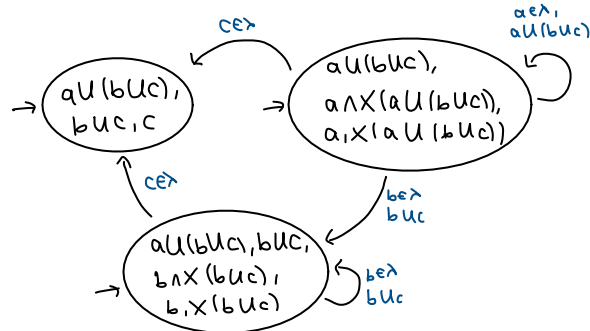


3. $aU(bUc)$,

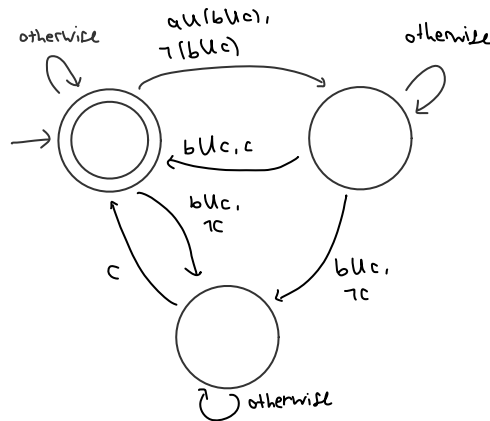
Solution: The steps are the same as for the previous example. First we process the formula and obtain the following SLBA:



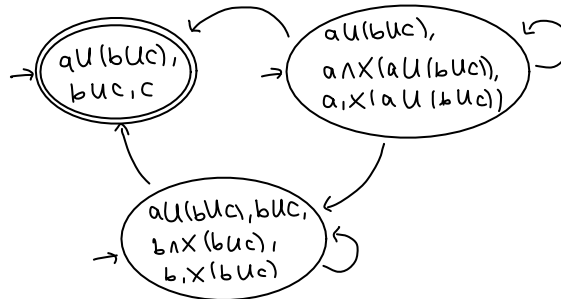
Then we keep only the real states (all states are accepting):



Then we consider the following eventuality automaton:

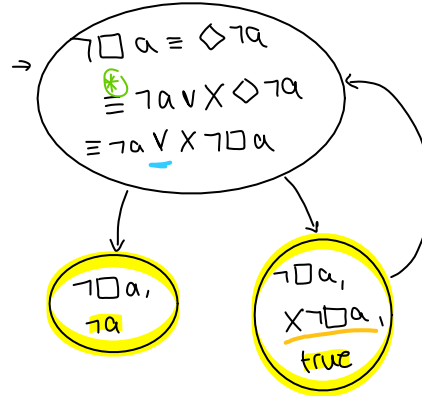


And finally we obtain the following SLBA, by intersecting the real states SLBA with the eventuality automaton:



4. $\neg \Box a$,

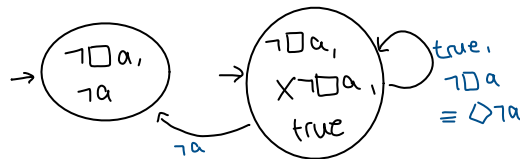
Solution: First we process the formula and obtain the following SLBA:



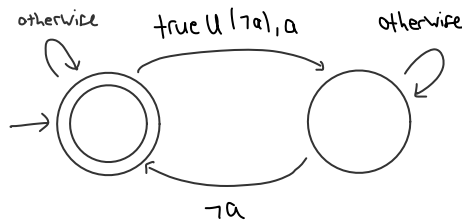
The second equivalence in the first state can be proved as follows:

$$\begin{aligned}
 \Diamond \neg a &\equiv \text{true } \mathcal{U} \neg a \\
 &\equiv \neg a \vee (\text{true} \wedge X(\text{true } \mathcal{U} \neg a)) \\
 &\equiv \neg a \vee (\text{true} \wedge X\Diamond \neg a) \\
 &\equiv \neg a \vee X\Diamond \neg a
 \end{aligned}$$

Then we keep only the real states (all states are accepting):



Then we consider the following eventuality automaton:



And finally we obtain the following SLBA, by intersecting the real states SLBA with the eventuality automaton:

